

Report on Re-implementing LoRA: Low-Rank Adaptation of Large Language Models

Julian Gasharov, jg2399

Link to GitHub Repository: <https://github.com/j2399/LoRA-reimp/tree/main>

1. Introduction

I re-implemented results from the paper [1], *LoRA: Low-Rank Adaptation of Large Language Models* by Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen.

Natural Language Processing utilizes adapting pretrained LLMs to multiple tasks by fine-tuning. Models these days are incredibly large, so full fine-tuning is infeasible because it updates ALL the parameters. The paper’s key insight is that the weight updates needed for task adaptation have low intrinsic rank, so you can freeze the pretrained weight matrix and inject two trainable small low-rank matrices, drastically reducing the number of trainable parameters while maintaining high accuracy.

2. Chosen Result

I aimed to reproduce their accuracy of 96.2 ± 0.5 on the SST-2 dataset when applying LoRA (0.8M trainable parameters) to RoBERTa-Large, as reported in Table 2 of [1] (Hu et al., 2022). This is within 0.5 of the accuracy of the full fine-tuned model (355M trainable parameters), showing LoRA achieves full accuracy with far fewer (about 0.2%) trainable parameters. I chose this result since it demonstrates LoRA’s effectiveness clearly and the dataset is straightforward to work with.

Model & Method	# Trainable Parameters	MNLI	SST-2
RoB _{base} (FT)*	125.0M	87.6	94.8
RoB _{base} (BitFit)*	0.1M	84.7	93.7
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$.0	94.2 $\pm$.1
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$.1	94.7 $\pm$.3
RoB _{base} (LoRA)	0.3M	87.5 $\pm$.3	95.1$\pm$.2
RoB _{large} (FT)*	355.0M	90.2	96.4
RoB _{large} (LoRA)	0.8M	90.6$\pm$.2	96.2 $\pm$.5

TABLE 1. Reference results from [1] (Hu et al., 2022) showing accuracy on SST-2.

3. Methodology

I fine-tune RoBERTa-Large [2] on the SST-2 sentiment classification task from the GLUE benchmark. The General Language Understanding Evaluation (GLUE) benchmark [3] is a collection of resources for training, evaluating, and analyzing natural language understanding systems; it includes the Stanford Sentiment Treebank (SST-2) [4], which is a corpus that allows for sentiment analysis. I took the data from HuggingFace [5].

I train for 6 epochs with batch size 32, learning rate 1×10^{-4} , AdamW optimizer [7] with weight decay 0.01, and a cosine schedule with 10% warmup. Gradient clipping is applied at norm 1.0. (Before the poster session, I trained for 3 epochs and learning rate 3×10^{-4} , due to computation constraints.) I run it all using PyTorch methods [6]. I inject LoRA adapters into the query and key matrices of every layer. I evaluate using classification accuracy on the SST-2 validation set. I evaluate the results

of the fine-tuned model and compare it to the accuracies in [1] (Hu et al., 2022) of the full fine-tuned and LoRA models.

The LoRA method reduces the number of trainable parameters by freezing the pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ and trains only two low-rank matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$, where $r \ll \min(d, k)$. The resulting weight update is $W_0 + BA$. In [1], the low-rank is $r = 8$.

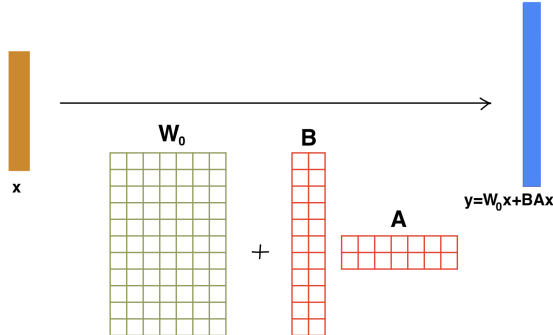


FIGURE 1. Forward pass in LoRA.

I explored my own modification of adding a third matrix $C \in \mathbb{R}^{r \times r}$ to the decomposition. The idea is to put a matrix C between the matrices B and A , so the update is $W_0 + BCA$; this allows the model to do some extra thinking in the low dimension, while very few trainable parameters are added. My thought process behind this is that using C might add accuracy because the low rank BA contains the important information for adaptation, so the extra computation BCA here is worth it.

4. Results & Analysis

The re-implementation achieved its top validation accuracy of 96.56% on SST-2 using low-rank $r = 4$. I also achieved 96.1% accuracy with $r = 8$. Both of these are within the 0.5 error range of the original paper’s [1] accuracy, which is $96.2_{\pm 0.5}\%$ for $r = 8$ (see Table 1). My results are presented in Table 2.

Model	Rank r	Trainable Params	Val Accuracy
LoRA re-implementation	4	395,266 (0.111%)	96.56%
LoRA re-implementation	8	788,482 (0.222%)	96.10%
LoRA with an additional matrix C	4	396,034 (0.111%)	95.41%
LoRA with an additional matrix C	8	791,554 (0.222%)	95.64%

TABLE 2. Results on SST-2 validation set (50% data, 6 epochs, seed 42).

These results confirm the claim in [1]: training with less than 0.25% of the parameters is enough to achieve near or full accuracy of fine-tuning on SST-2.

Previously (before the poster session), training for fewer epochs (3 epochs) due to computation constraints and with a random seed, led to the model underperforming with two matrices A, B (validation accuracy 95.06% with $r = 4$) but performing better with 3 matrices A, B, C (validation accuracy 95.87% with $r = 4$). In contrast, training for 6 epochs and seed 42, leads to lower accuracy with 3 matrices compared to with 2 matrices (see Table 2). I believe this is because of the randomness of the seed, and the 2 matrices taking longer but training more stably. It is also possible that this is due to the problem not being complex enough so the extra matrix just impedes gradient flow rather than doing useful work. More tests on other datasets would be needed to see if/when adding the third matrix is effective.

5. Reflections

I learned how to replace the layers in RoBERTa-Large with LoRA layers. The key takeaway is that LoRA adapters drastically reduce the number of trainable parameters, while maintaining high accuracy, allowing the model to be fine-tuned with much lower storage and compute costs.

Some Potential Future Directions for Research are:

- understand why the mechanism of LoRA works;
- I discovered that adding a third matrix (in the decomposition) can in some cases improve performance, and that there is room for more research in this area;
- combine LoRA with other adaptation methods.

REFERENCES

- [1] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022. [arXiv:2106.09685](#)
- [2] Liu, Y., Ott, M., Goyal, N., et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. [arXiv:1907.11692](#), 2019.
- [3] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S.R. *GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding*. EMNLP BlackboxNLP Workshop, 2018.
- [4] Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C.D., Ng, A., and Potts, C. *Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank*. EMNLP, pp. 1631–1642, 2013.
- [5] Wolf, T., Debut, L., Sanh, V., et al. *Transformers: State-of-the-Art Natural Language Processing*. EMNLP System Demonstrations, pp. 38–45, 2020.
- [6] Paszke, A., Gross, S., Massa, F., et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.
- [7] Loshchilov, I. and Hutter, F. *Decoupled Weight Decay Regularization*. ICLR, 2019. [arXiv:1711.05101](#)